LAB ASSIGNMENT 7-10

```
CREATE TABLE Department ( dept_id INT
        PRIMARY KEY, dept_name
        VARCHAR(50)
);

INSERT INTO Department VALUES (10,
'Computer Science'),
(20, 'Electronics'),
(30, 'Mechanical'),
(40, 'Civil');

CREATE TABLE Student ( student_id INT
        PRIMARY KEY, name VARCHAR(50),
        dept_id INT
);

INSERT INTO Student VALUES (1,
'Rahul', 10),
(2, 'Priya', 20),
(3, 'Aman', 10),
(4, 'Neha', 30),
(5, 'Karan', NULL);

SELECT Student.student_id, Student.name, Department.dept_name FROM Student
INNER JOIN Department
ON Student.dept_id = Department.dept_id;

SELECT Student.student_id, Student.name, Department.dept_name FROM Student
LEFT OUTER JOIN Department
ON Student.dept_id = Department.dept_id;

SELECT Student.student_id, Student.name, Department.dept_name FROM Student
RIGHT OUTER JOIN Department
ON Student.dept_id = Department.dept_id;
```

```
SELECT Student.student_id, Student.name, Department.dept_name FROM Student
LEFT JOIN Department
ON Student.dept_id = Department.dept_id UNION
SELECT Student.student_id, Student.name, Department.dept_name FROM Student
RIGHT JOIN Department
ON Student.dept_id = Department.dept_id;

SELECT *
FROM Student
NATURAL JOIN Department;

CREATE TABLE Employee ( emp_id
        INT PRIMARY KEY, name
        VARCHAR(50),
        departmentVARCHAR(50), salary INT
);

INSERT INTO Employee VALUES (1,
'Rahul', 'IT', 50000),
(2, 'Priya', 'HR', 40000),
(3, 'Aman', 'IT', 60000),
(4, 'Neha', 'HR', 45000),
(5, 'Karan', 'Sales', 35000),
(6, 'Riya', 'IT', 55000);

SELECT department, COUNT(*) AS employee_count FROM Employee
GROUP BY department;

SELECT department, AVG(salary) AS average_salary FROM Employee
GROUP BY department;

SELECT department, SUM(salary) AS total_salary FROM Employee
```

```
GROUP BY department;

SELECT department, COUNT(*) AS employee_count FROM Employee
GROUP BY department HAVING
COUNT(*) > 1;

SELECT department, AVG(salary) AS average_salary FROM Employee
GROUP BY department
HAVING AVG(salary) > 40000;

SELECT *
FROM Employee
ORDER BY salary ASC;

SELECT *
FROM Employee
ORDER BY salary DESC;

SELECT *
FROM Employee ORDER
BY name ASC;

SELECT department,
        COUNT(*) AS employee_count, AVG(salary) AS
        average_salary
FROM Employee
GROUP BY department
HAVING AVG(salary) > 40000
ORDER BY average_salary DESC;
```

OUPUT :

Output:

```
+------------+-------+------------------+
| student_id | name  | dept_name        |
+------------+-------+------------------+
|          1 | Rahul | Computer Science |
|          2 | Priya | Electronics      |
|          3 | Aman  | Computer Science |
|          4 | Neha  | Mechanical       |
+------------+-------+------------------+
+------------+-------+------------------+
| student_id | name  | dept_name        |
+------------+-------+------------------+
|          1 | Rahul | Computer Science |
|          2 | Priya | Electronics      |
|          3 | Aman  | Computer Science |
|          4 | Neha  | Mechanical       |
|          5 | Karan | NULL             |
+------------+-------+------------------+
+------------+-------+------------------+
| student_id | name  | dept_name        |
+------------+-------+------------------+
|          3 | Aman  | Computer Science |
|          1 | Rahul | Computer Science |
|          2 | Priya | Electronics      |
|          4 | Neha  | Mechanical       |
|       NULL | NULL  | Civil            |
+------------+-------+------------------+
```

Output:

student_id	name	dept_name
1	Rahul	Computer Science
2	Priya	Electronics
3	Aman	Computer Science
4	Neha	Mechanical
5	Karan	NULL
NULL	NULL	Civil

dept_id	student_id	name	dept_name
10	1	Rahul	Computer Science
20	2	Priya	Electronics
10	3	Aman	Computer Science
30	4	Neha	Mechanical

department	employee_count
IT	3
HR	2
Sales	1

```
+------------+----------------+
| department | average_salary |
+------------+----------------+
| IT         |     55000.0000 |
| HR         |     42500.0000 |
| Sales      |     35000.0000 |
+------------+----------------+
```

```
+------------+--------------+
| department | total_salary |
+------------+--------------+
| IT         |       165000 |
| HR         |        85000 |
| Sales      |        35000 |
+------------+--------------+
```

```
+------------+----------------+
| department | employee_count |
+------------+----------------+
| IT         |              3 |
| HR         |              2 |
+------------+----------------+
```

```
+------------+----------------+
| department | average_salary |
+------------+----------------+
| IT         |     55000.0000 |
| HR         |     42500.0000 |
+------------+----------------+
```

```
+--------+-------+------------+--------+
| emp_id | name  | department | salary |
+--------+-------+------------+--------+
|      5 | Karan | Sales      |  35000 |
|      2 | Priya | HR         |  40000 |
|      4 | Neha  | HR         |  45000 |
|      1 | Rahul | IT         |  50000 |
|      6 | Riya  | IT         |  55000 |
|      3 | Aman  | IT         |  60000 |
+--------+-------+------------+--------+
```

```
+--------+-------+------------+--------+
| emp_id | name  | department | salary |
+--------+-------+------------+--------+
|      3 | Aman  | IT         |  60000 |
|      6 | Riya  | IT         |  55000 |
|      1 | Rahul | IT         |  50000 |
|      4 | Neha  | HR         |  45000 |
|      2 | Priya | HR         |  40000 |
|      5 | Karan | Sales      |  35000 |
+--------+-------+------------+--------+
```

```
+--------+-------+------------+--------+
+--------+-------+------------+--------+
| emp_id | name  | department | salary |
+--------+-------+------------+--------+
|      3 | Aman  | IT         |  60000 |
|      5 | Karan | Sales      |  35000 |
|      4 | Neha  | HR         |  45000 |
|      2 | Priya | HR         |  40000 |
|      1 | Rahul | IT         |  50000 |
|      6 | Riya  | IT         |  55000 |
+--------+-------+------------+--------+
+------------+----------------+----------------+
| department | employee_count | average_salary |
+------------+----------------+----------------+
| IT         |              3 |     55000.0000 |
| HR         |              2 |     42500.0000 |
+------------+----------------+----------------+
```

LAB 9 AND 10

```
CREATE TABLE Employee ( emp_id
        INT PRIMARY KEY, name
        VARCHAR(50),
        departmentVARCHAR(50), salary INT
);

INSERT INTO Employee VALUES (1,
'Rahul', 'IT', 50000),
(2, 'Priya', 'HR', 40000),
(3, 'Aman', 'IT', 60000),
(4, 'Neha', 'HR', 45000),
(5, 'Karan', 'Sales', 35000); DELIMITER //

CREATE PROCEDURE ShowEmployees()
BEGIN
        SELECT * FROM Employee; END
//

DELIMITER ;

CALL ShowEmployees(); DELIMITER //

CREATE PROCEDURE AddEmployee( IN
        p_id INT,
        IN p_name VARCHAR(50),
        IN p_department VARCHAR(50), IN
        p_salary INT
)
BEGIN
        INSERT INTO Employee
        VALUES (p_id, p_name, p_department, p_salary); END //

DELIMITER ;
```

```
CALL AddEmployee(6, 'Arjun', 'IT', 52000); DELIMITER //

CREATE PROCEDURE UpdateEmployee( IN
    p_id INT,
    IN p_salary INT
)
BEGIN
    UPDATE Employee
    SET salary = p_salary WHERE
    emp_id = p_id;
END // DELIMITER ;

CALL UpdateEmployee(1, 55000);

DELIMITER //

CREATE PROCEDURE DeleteEmployee( IN
    p_id INT
)
BEGIN
    DELETE FROM Employee
    WHERE emp_id = p_id;
END // DELIMITER ;

CALL DeleteEmployee(6);

CREATE TABLE Employee_Log (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    message VARCHAR(100),
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //
```

```
CREATE TRIGGER before_employee_insert BEFORE
INSERT ON Employee
FOR EACH ROW
BEGIN
        IF NEW.salary < 0 THEN SET
                NEW.salary = 0;
        END IF;
END // DELIMITER

; DELIMITER //

CREATE TRIGGER after_employee_insert AFTER
INSERT ON Employee
FOR EACH ROW
BEGIN
        INSERT INTO Employee_Log(message) VALUES
        ('Employee inserted');
END // DELIMITER

; DELIMITER //

CREATE TRIGGER before_employee_update BEFORE
UPDATE ON Employee
FOR EACH ROW
BEGIN
        IF NEW.salary < 0 THEN SET
                NEW.salary = 0;
        END IF;
END // DELIMITER

; DELIMITER //

CREATE TRIGGER after_employee_update
```

```
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Employee_Log(message) VALUES
    ('Employee updated');
END // DELIMITER

; DELIMITER //

CREATE TRIGGER before_employee_delete BEFORE
DELETE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.emp_id = 1 THEN SIGNAL
        SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee cannot be deleted'; END IF;
END // DELIMITER

; DELIMITER //

CREATE TRIGGER after_employee_delete AFTER
DELETE ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Employee_Log(message) VALUES
    ('Employee deleted');
END // DELIMITER ;

INSERT INTO Employee VALUES (7,
'Rohan', 'HR', -5000);

UPDATE Employee SET
salary = -1000
```

```
WHERE emp_id = 2;

UPDATE Employee SET
salary = 60000 WHERE
emp_id = 3;

DELETE FROM Employee
WHERE emp_id = 7;

SELECT * FROM Employee;

SELECT * FROM Employee_Log;
```

OUPUT:

```
+--------+-------+------------+--------+
| emp_id | name  | department | salary |
+--------+-------+------------+--------+
|      1 | Rahul | IT         |  50000 |
|      2 | Priya | HR         |  40000 |
|      3 | Aman  | IT         |  60000 |
|      4 | Neha  | HR         |  45000 |
|      5 | Karan | Sales      |  35000 |
+--------+-------+------------+--------+
+--------+-------+------------+--------+
| emp_id | name  | department | salary |
+--------+-------+------------+--------+
|      1 | Rahul | IT         |  55000 |
|      2 | Priya | HR         |      0 |
|      3 | Aman  | IT         |  60000 |
|      4 | Neha  | HR         |  45000 |
|      5 | Karan | Sales      |  35000 |
+--------+-------+------------+--------+
+--------+-------------------+---------------------+
| log_id | message           | log_time            |
+--------+-------------------+---------------------+
|      1 | Employee inserted | 2026-10-04 08:51:47 |
|      2 | Employee updated  | 2026-10-04 08:51:47 |
|      3 | Employee updated  | 2026-10-04 08:51:47 |
|      4 | Employee deleted  | 2026-10-04 08:51:47 |
+--------+-------------------+---------------------+
```